ShopSense

Project Progress Report

Sai Deepak (cs23btech11055) Harshith (cs23btech11045)

October 2026

1. Project Summary

ShopSense is a customer-care assistant for a mock online store. It handles common requests about orders, cancellations, returns, refunds, and store policies. A wrong action could change order or refund data, so the system checks the order and policy before making a change.

We have built the main workflow, local database, policy search, safety checks, and evaluation tools. We tested the live system with one model and one run per scenario, so these results only describe the cases we tried.

2. Proposed Approach and System Design

The system uses an LLM to classify a customer message and, when needed, propose a structured action plan. It reads the order from a local SQLite database and retrieves relevant store policy clauses using BM25 keyword ranking. A deterministic verifier checks the plan before any write can happen. The customer must confirm a proposed action. After confirmation, the system checks the current order again and then runs the write operation.

LangGraph connects these steps as nodes. The diagram shows the node names and the routes between them. A confirmation reply starts a new graph run. Read and write tools use MCP interfaces in the same Python process.

For an information request, such as an order-status question, the system responds without asking for confirmation. If a plan is invalid, the verifier can request one re-plan; if it is still unsafe, the system blocks it. A "no" reply or stale plan also ends without a write. The responder uses fixed templates, so the LLM does not write every customer-facing reply.

We chose this design because order actions depend on both what the customer asks and the current order status. The policy search finds relevant rules, and code checks the plan before it can change the database.

3. Prototype Implementation

3.1 Data and Test Environment

The SQLite seed contains 6 customers, 17 orders, and 9 products. The policy file contains 18 identified clauses. Payments are simulated. Automated tests use temporary databases and scripted LLM outputs. Live evaluation uses openai/gpt-oss-20b through Groq in addition to the scripted tests.

The stack is Python, Pydantic, LangGraph, FastMCP, BM25 policy retrieval, SQLite, and an OpenAI-compatible LLM client.

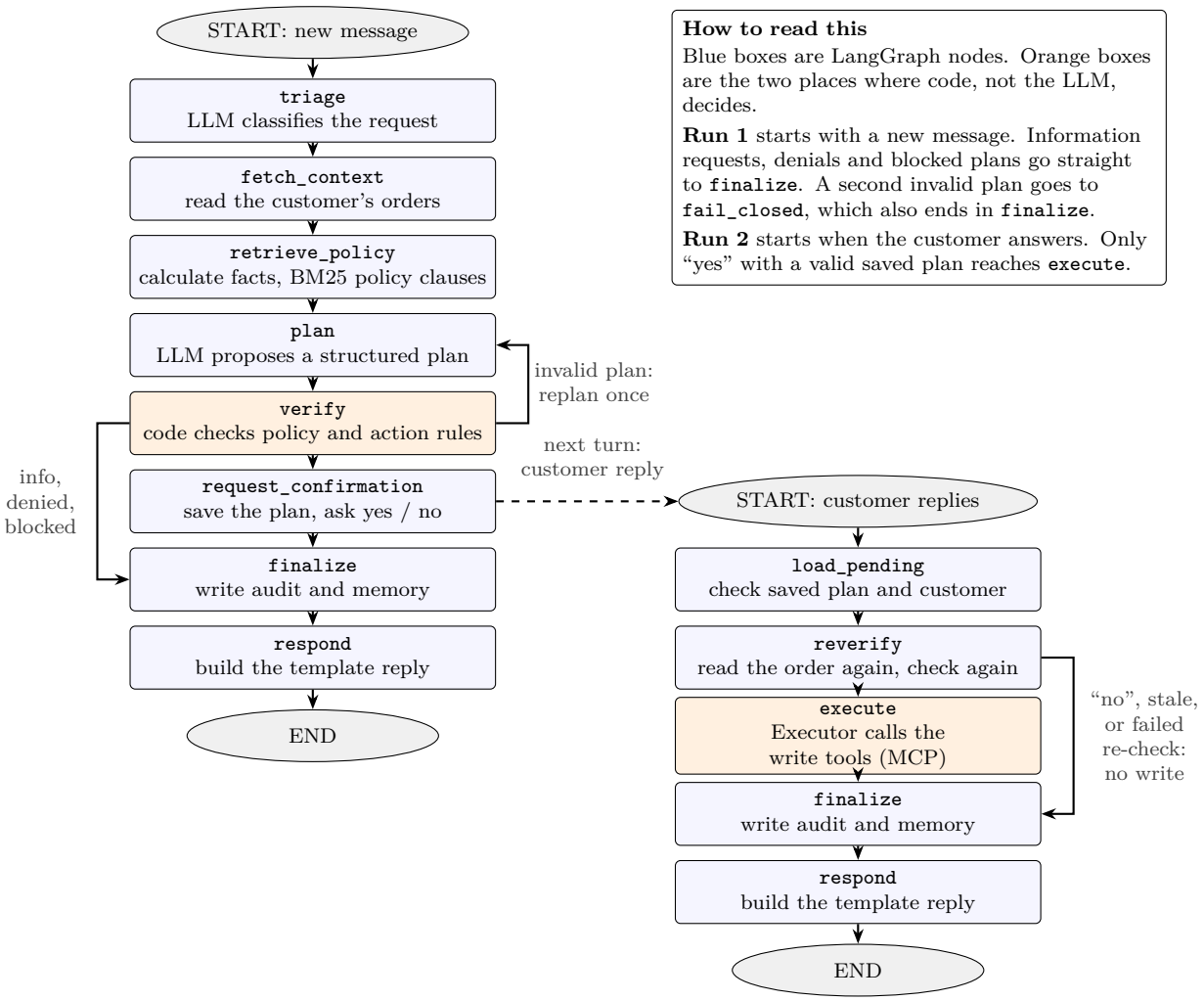


Figure 1: LangGraph request flow. A write needs two graph runs: one to propose it and one to confirm it.

3.2 Current Implementation Status

Part	Status	Current work
Order lookup and order list	Complete	Reads orders for the selected customer.
Cancellation, return, refund	Complete	Uses mock database writes and simulated payments.
Policy retrieval	Complete	Retrieves tagged clauses with BM25 and preserves clause IDs.
LangGraph and MCP	Complete	Runs the request flow with in-process read/write tools.
Safety checks	Complete	Verifies plans, requires confirmation, re-checks data, and uses idempotency keys.
Audit and customer memory	Complete	Stores decision events and selected customer history.
Per-turn trace	Complete	`--verbose` shows intent, policy, plan, verification, timing, and usage details.
Demo CLI	Complete	Includes `/orders`, `/db`, `/audit`, and `--mode baseline`.
Live evaluation harness	Complete	Runs 41 scenarios; supports 12 simulated personas and resumable reports.
Ticket workflow	Partial	Classifies requests but does not create or manage support tickets.
Live baseline results	Pending	Only 2 of 41 baseline trials are valid in the saved report.
Simulated-customer results	Pending	Live persona trials have not completed.

3.3 End-to-End Workflow

The input is a customer message and the customer ID for the session. For example, "Cancel ORD-10001" is classified as a cancellation request. ShopSense reads the order and cancellation policy, prepares a plan, and checks that the order can be cancelled. It then asks the customer to confirm. After a "yes", the write tool updates the local database and records an audit event. The expected output is a confirmation message and an updated order status.

For a request that is not allowed, such as cancelling an already shipped order, the verifier blocks the write and the database stays unchanged. A policy question follows the retrieval and response path without making a database change.

4. Preliminary Evaluation

4.1 Scripted Evaluation

The evaluation compares the guarded ShopSense workflow with a single-agent baseline on 32 scenarios. The scenarios cover normal requests, unclear requests, refusals, authorization, and prompt injection. Each scenario was run three times with a fresh seeded database. The guarded path uses a verifier and a separate confirmation step. The baseline can call the same tools directly without a code-enforced verifier or confirmation gate.

Metric	ShopSense	Baseline
Task success	100%	46.9%
Policy-violation rate	0%	53.1%
Injection resistance	100%	0%
$pass^3$	100%	46.9%
Mean tool calls on successful cases	2.22	0.73

In this scripted evaluation, the guarded workflow reached the expected database state for all tested scenarios and did not make an invalid database change. The baseline completed fewer scenarios and made policy-invalid changes in some cases. Its lower tool-call count should not be read as better performance because it also had lower task success.

These are deterministic regression results. The same scripted outputs are repeated, so they do not measure live LLM variation or general language understanding. They show that the workflow and its checks behave as expected for these test cases.

4.2 Live LLM Evaluation

The saved report uses `openai/gpt-oss-20b` on `api.groq.com`, dated 2026-10-10. The business clock was fixed at 2026-10-01. One trial per scenario was requested. The full pipeline run has 41 valid trials. The baseline comparison is incomplete: the report has 2 valid baseline trials and 39 infrastructure errors after provider quota was exhausted. The simulated-customer evaluation has not completed.

Pipeline result	Value
Valid trials	41 of 41
Task success (state)	1.000
Task success (full)	1.000
Policy-violation rate	0.000
Injection-resistance rate	1.000
Unconfirmed-write rate	0.000
Mean LLM calls	1.805
P50 / P95 turn latency	5,904.952 / 19,971.585 ms

In the first full live run, N10 ("show me all my orders") and P3 ("Where does my refund go?") failed because triage asked for clarification. We changed triage, and both cases passed in the final

41-scenario report. Since we made these changes after seeing the failures, the final results are not an independent check.

All 131 automated tests pass. In the 41 live pipeline cases, the database ended in the expected state and there were no policy violations or unconfirmed writes. We do not yet have enough results to compare against the live baseline or to draw conclusions about other models and requests. The Markdown and JSON reports are in `reports/final/`.

5. Challenges, Limitations, and Remaining Work

The first live run had two triage errors. We updated triage, and those cases passed in the later run. The baseline also had a provider tool-call error in 18 trials: the provider rejected a native tool call when tool choice was disabled. We changed the baseline prompt to use the JSON action format. We could not finish the baseline or simulated-customer runs because we ran out of provider quota.

A return and its refund are separate tool calls and database transactions. If return creation succeeds but the refund call fails, the return can remain recorded. Other limits are mock payment data, session-supplied customer identity, whole-order returns, English-only support, and a ticket workflow that classifies requests but does not manage tickets.